Firebase Authentication y Cloud Firestore aplicados a Panadería Rosario

Avance 2 — Foro de Discusión 2

Investigación técnica y aplicación en una app Android con Kotlin y Jetpack Compose

Proyecto: GabrielQuintanilla94/Panaderia

3 de noviembre de 2026

Documento técnico del proyecto. Los datos académicos personales y de la institución pueden agregarse antes de la entrega.

Resumen

Este documento analiza la integración de Firebase Authentication y Cloud Firestore en Panadería Rosario. La aplicación original empleaba identificación local por teléfono y datos en memoria. La integración incorpora identidad mediante correo y Google, persistencia de perfiles y pedidos vinculados al identificador del usuario, y actualización del historial mediante listeners. Se distinguen la implementación del cliente Android y la activación de los servicios en una cuenta Firebase, que debe completar el propietario. No se presentan pruebas remotas como realizadas sin un proyecto configurado.

Problema y objetivo

Los datos en memoria se pierden al terminar el proceso y no pueden compartirse entre dispositivos. Identificar un cliente únicamente por su teléfono tampoco demuestra que quien lo escribe sea su propietario. El objetivo es autenticar a cada persona y conservar sus registros en una base remota, manteniendo catálogo, carrito, administración y reportes.

Firebase como plataforma

Firebase reúne servicios administrados para desarrollo móvil. Authentication gestiona cuentas y credenciales, mientras Firestore conserva documentos y colecciones. Son servicios complementarios: iniciar sesión no concede acceso universal a la base; las reglas deben autorizar cada operación según la identidad y el recurso solicitado (Google, s. f.-a; Google, s. f.-e).

Autenticación por correo y Google

El registro con correo y contraseña se implementa mediante createUserWithEmailAndPassword; el acceso posterior utiliza signInWithEmailAndPassword. La contraseña se entrega al SDK de Authentication y no se guarda en documentos de Firestore. El código incorpora estados de espera y errores de autenticación. La validación local de longitud ayuda a orientar al usuario, pero la política definitiva la aplica Firebase (Google, s. f.-b).

El acceso de Google utiliza Android Credential Manager para obtener un ID token. GoogleAuthProvider transforma ese token en una credencial para Firebase. El resultado es un usuario de Firebase con UID, que permite consultar sus documentos sin depender del número telefónico. Es necesario registrar el certificado Android y utilizar el identificador OAuth de cliente web; no debe reemplazarse por el identificador del cliente Android (Google, s. f.-c).

Identidad y permisos administrativos

Los registros del cliente se asocian con FirebaseAuth.currentUser.uid. Un campo de perfil no convierte al cliente en administrador. La aplicación consulta el custom claim admin, que solo se asigna desde un entorno administrativo confiable. Esto reemplaza las credenciales locales de demostración y permite que las reglas de Firestore verifiquen el mismo permiso (Google, s. f.-d).

Ciclo de sesión

AppState se conserva como ViewModel. Al iniciar se recupera la sesión existente y se suscribe a los datos. Al cerrar sesión se eliminan listeners y se vacía el historial local y el carrito. La app no presenta como propios los pedidos de una sesión anterior. El acceso administrativo requiere seleccionar esa modalidad al ingresar.

Modelo de datos en Firestore

La colección usuarios usa el UID como identificador de documento y contiene nombre, correo, teléfono y dirección. productos mantiene catálogo y existencias compartidas. pedidos incluye usuarioId, datos del receptor, detalles de productos, cantidades, total, estado y fecha del servidor. contadores/pedidos permite asignar una numeración legible junto con la referencia documental. Las contraseñas y claves privadas no forman parte de este modelo.

El historial del cliente consulta pedidos con la condición usuarioId igual al UID autenticado. Los administradores pueden consultar todos los pedidos para dar seguimiento, coordinar entregas y elaborar reportes. Los listeners actualizan las pantallas cuando cambia un documento. El carrito sigue siendo temporal; el pedido confirmado constituye el registro persistente.

Consistencia de pedidos

La confirmación utiliza una transacción que lee el contador y los productos, verifica existencias y precios, descuenta stock y crea el pedido. Si hay un conflicto concurrente, Firestore puede volver a ejecutar la transacción; por eso no se modifican estados de interfaz dentro del bloque transaccional. El carrito se limpia después de que la escritura termina correctamente (Google, s. f.-f).

Los estados mantienen el orden RECIBIDO, EN_PREPARACION, EN_CAMINO y ENTREGADO. La actualización administrativa comprueba el estado remoto antes de avanzar. Las reglas autorizan únicamente la transición siguiente e impiden que un cliente cambie el estado de entrega de su pedido.

Seguridad, límites y responsabilidades

Las reglas incluidas restringen perfiles al propietario y pedidos al propietario o al administrador. El catálogo requiere autenticación para lectura y permisos administrativos para su gestión. El descuento de stock referencia un nuevo pedido del usuario dentro de la misma operación. Las reglas deben desplegarse explícitamente: tener el archivo en Git no protege por sí solo una base remota (Google, s. f.-e).

Una aplicación comercial debe calcular y validar el importe definitivo desde un backend confiable. El prototipo conserva comprobaciones de precio en el cliente y restricciones de datos en reglas, pero estas no equivalen a una validación integral de cobros contra solicitudes manipuladas. Tampoco se incorpora pago electrónico. La edición administrativa del catálogo completo se plantea para un administrador a la vez.

Validación y demostración

Las pruebas JVM verifican totales, serialización de productos, historial y estados. Compilación y Android Lint verifican el cliente; no sustituyen las pruebas remotas de Firebase.

La demostración debe incluir correo, Google, catálogo, confirmación y recuperación de pedidos tras reiniciar. Una segunda cuenta debe ver solo sus registros. El administrador avanzará el estado y el cliente observará el cambio. Se requieren proyecto Firebase, reglas publicadas y dispositivo compatible.

Conclusión

Firebase incorpora identidad y persistencia al proyecto sin reemplazar sus módulos. La integración requiere configurar el proyecto remoto y verificar los flujos con cuentas reales antes de declarar operativa la conexión en la nube.

Referencias

Google. (s. f.-a). *Agrega Firebase a tu proyecto de Android*. Firebase. https://firebase.google.com/docs/android/setup

Google. (s. f.-b). *Autentica con Firebase mediante cuentas basadas en contraseña en Android*. Firebase. https://firebase.google.com/docs/auth/android/password-auth

Google. (s. f.-c). *Autentica mediante Acceso con Google en Android*. Firebase. https://firebase.google.com/docs/auth/android/google-signin

Google. (s. f.-d). *Controla el acceso con atributos personalizados y reglas de seguridad*. Firebase. https://firebase.google.com/docs/auth/admin/custom-claims

Google. (s. f.-e). *Escribe condiciones para las reglas de seguridad de Cloud Firestore*. Firebase. https://firebase.google.com/docs/firestore/security/rules-conditions

Google. (s. f.-f). *Transacciones y escrituras en lotes*. Firebase. https://firebase.google.com/docs/firestore/manage-data/transactions

Nota. Se emplea autor corporativo y fecha no especificada para documentación técnica que se actualiza de forma continua. Los títulos se presentan en español para facilitar su identificación. Las referencias remiten a las fuentes oficiales de los servicios aplicados.